



Université Saad Dahlab de Blida
Faculté des sciences
Département de mathématiques



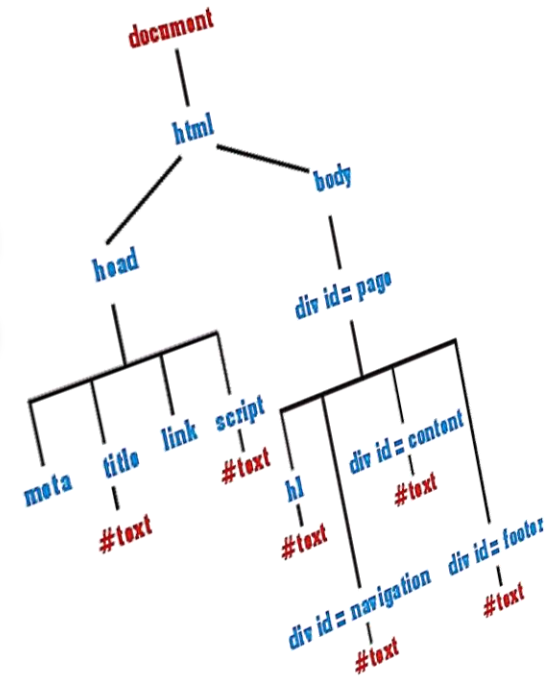
Programmation Avancée pour le Web

3^{ème} Année Licence

JavaScript



JavaScript et le DOM



BENAISSI Sellami

hayat.daoud@gmail.com

2024-2025

1. **Definition DOM**
2. **La structure d'une page web**
3. **L'objet Document**
4. **Accéder a un élément HTML**
5. **Modifier du contenu HTML**

- 1. [Définition DOM](#)
- 2. [La structure d'une page web](#)
- 3. [L'objet Document](#)
- 4. [Accéder à un élément HTML](#)
- 5. [Modifier du contenu HTML](#)

Definition DOM



DOM : **D**ocument **O**bject **M**odel

Le **DOM** est une norme du W3C, permet aux programmes et scripts **d'accéder** et de **modifier dynamiquement** le **contenu**, la **structure** et le **style** de documents XML ou HTML.



Le Document Object Model d'une page web va être créé automatiquement par le navigateur lors du chargement de la page.

Definition DOM

Definition



Le **DOM** est l'interface programmatique (API) qui permet au développeur web d'accéder et de manipuler le contenu d'une page web.

- 1 . Il fournit une **représentation structurée** et orientée objet des éléments et du contenu d'une page avec les méthodes permettant de modifier les propriétés de ces objets.

```
document.getElementById('texte').textContent = '<span>Texte  
modifié</span>';
```

```
document.body.style.color = 'blue';
```

```
document.querySelector("p a").href='http://www.mypage.dz';
```

Definition DOM

Definition



- 2 . Il fournit aussi des méthodes permettant l'ajout et la suppression de tels objets, permettant ainsi au développeur web de créer du contenu dynamique en modifiant le contenu, la structure et le style du document.

```
let newP = document.createElement('p');  
let newTexte = document.createTextNode('Texte écrit en JavaScript');
```

Definition DOM

Definition



- 3 . Il fournit aussi une interface de **gestion des événements**, pour capturer les actions du navigateur et de son utilisateur, et d'y réagir.

```
let b1 = document.querySelector('button');
```

```
b1.onclick = function(){alert('Bouton cliqué')};
```

```
b1.addEventListener('click', function(){alert('Bouton cliqué')});
```

- 1. [Définition DOM](#)
- 2. [La structure d'une page web](#)
- 3. [L'objet Document](#)
- 4. [Accéder à un élément HTML](#)
- 5. [Modifier du contenu HTML](#)

Definition DOM



DOM : **D**ocument **O**bject **M**odel

Le DOM étant une API, il est utilisé depuis un langage de programmation qui s'exécute dans le navigateur (eg: JavaScript).



le DOM a été conçu pour être indépendant de tout langage.

- 1. [Définition DOM](#)
- 2. [La structure d'une page web](#)
- 3. [L'objet Document](#)
- 4. [Accéder à un élément HTML](#)
- 5. [Modifier du contenu HTML](#)

La structure d'une page web



Une page web n'est rien d'autre qu'un ensemble de balises imbriquées les unes dans les autres. On peut la représenter sous une forme hiérarchisée appelée

une arborescence.

L'élément `<html>` en constitue la racine et contient deux éléments : `<head>` et `<body>`, qui contiennent eux-mêmes plusieurs sous-éléments.

La structure d'une page web

Tout dans la page HTML va être considéré comme
un nœud « node » :

- le **document** HTML lui même,
- les **éléments** HTML,
- les **attributs** HTML,
- le **texte** à l'intérieur des éléments,
- etc...

La structure d'une page web

Il existe différents types de **nœuds**. Dans un document HTML ordinaire existent dans tous les cas trois types de nœud importants qu'il nous faut distinguer :

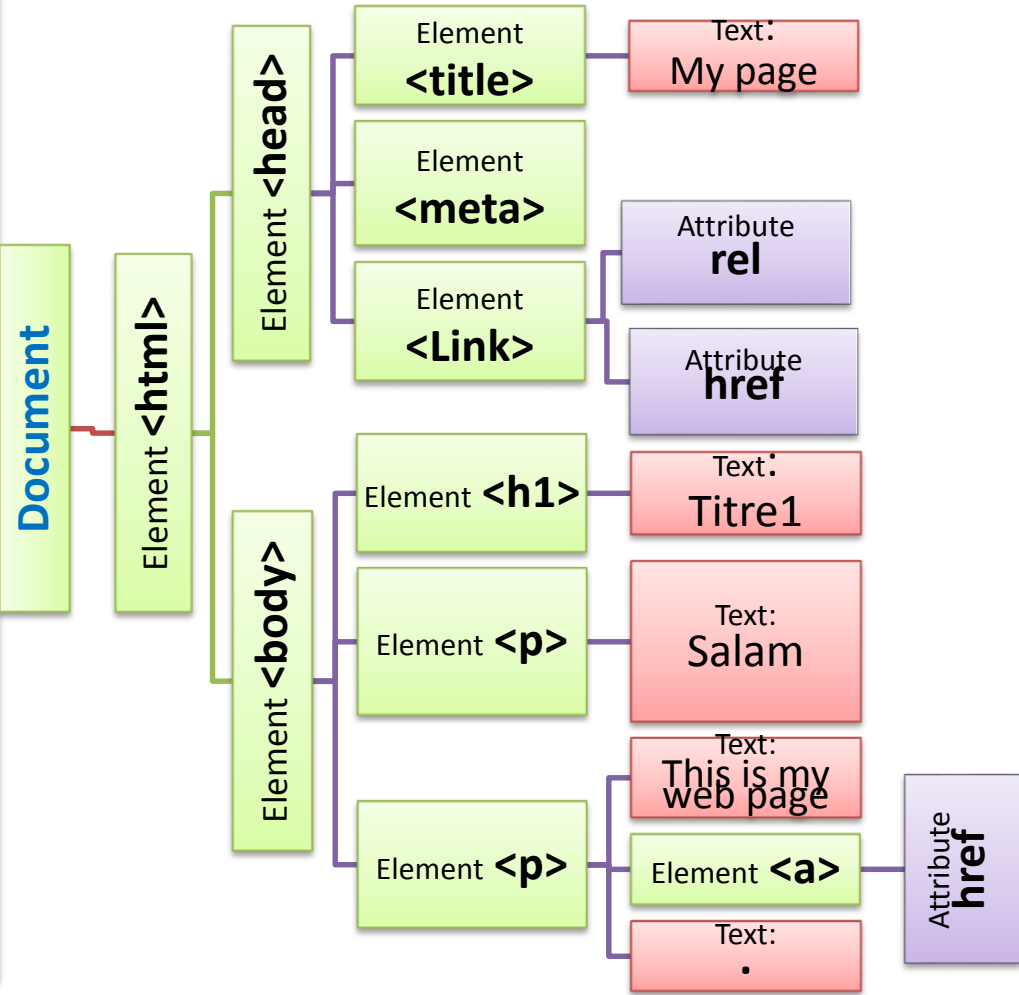
- les **nœuds-élément** (`element_node`),
- les **nœuds-attribut** (`attribute_node`),
- et les **nœuds-texte** (`text_node`).

- 1. Définition DOM
- 2. La structure d'une page web
- 3. L'objet Document
- 4. Accéder a un élément HTML
- 5. Modifier du contenu HTML

La structure d'une page web

Example

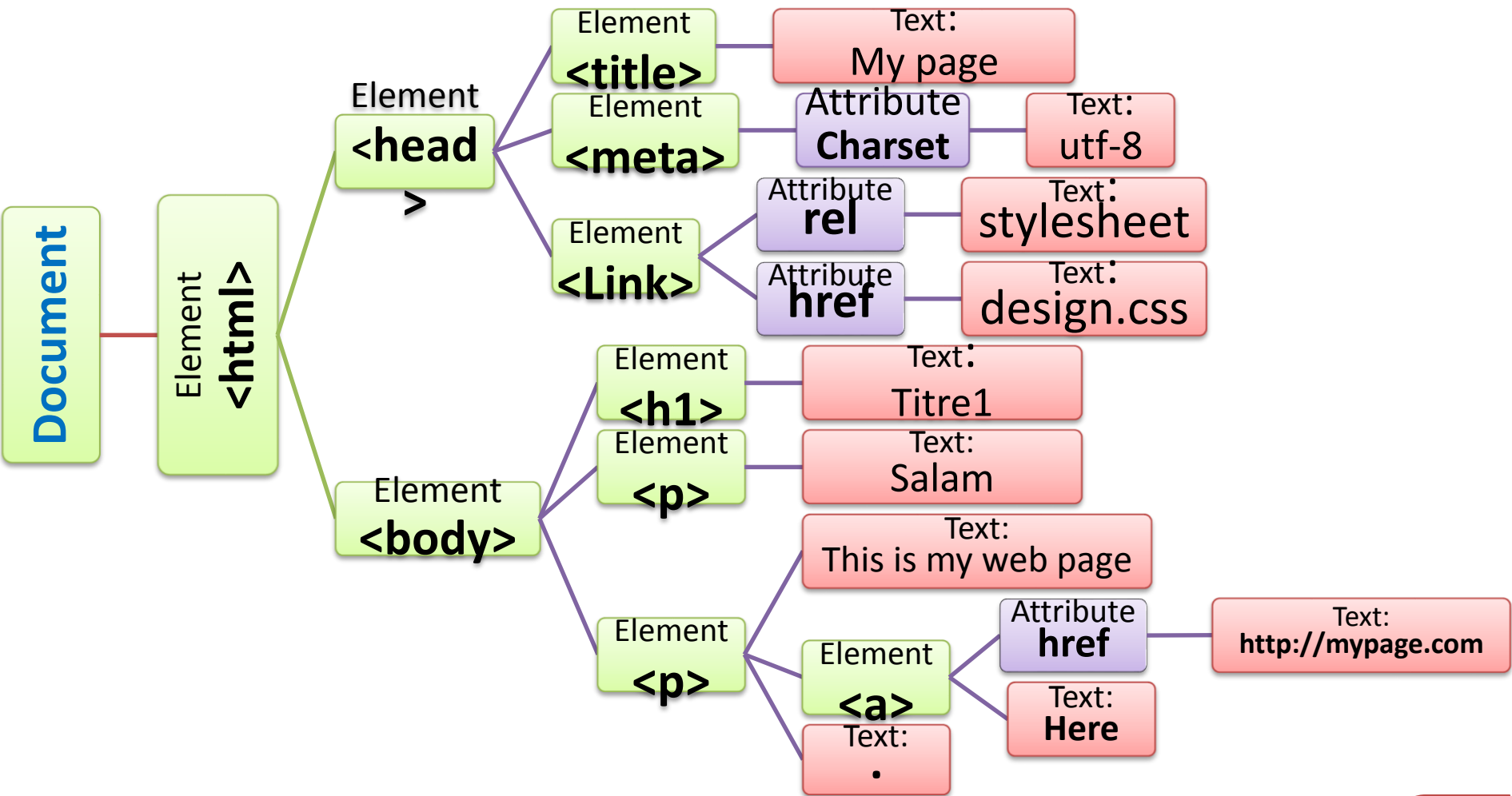
```
<html>
<head>
  <title> My page </title>
  <meta charset="utf-8" />
  <link rel="stylesheet"
href="design.css" />
</head>
<body>
  <h1> Titre1</h1>
  <p> Salam,</p>
  <p> This is my web page <a
href="http://mypage.com">Here</a
>.</p>
</body>
</html>
```



- 1. Définition DOM
- 2. La structure d'une page web
- 3. L'objet Document
- 4. Accéder à un élément HTML
- 5. Modifier du contenu HTML

La structure d'une page web

Et voici le DOM relatif à cette page qui va être créé par le navigateur :



- 1. [Définition DOM](#)
- 2. [La structure d'une page web](#)
- 3. [L'objet Document](#)
- 4. [Accéder à un élément HTML](#)
- 5. [Modifier du contenu HTML](#)

L'objet Document

L'objet document est un objet important, qui représente l'ensemble de l'arborescence du document analysé. Il possède de nombreuses propriétés et méthodes.

le document HTML en soi est représenté dans le DOM par un objet **Document**. **L'objet Document**, comme tous les autres objets créés par le DOM, est un nœud.

Cet objet Document va donc représenter notre page web en soi et disposer de méthodes nous permettant de modifier cette page web.

L'objet Document va également contenir tous les autres objets créés par le DOM, comme par exemple l'objet Element.

Accéder a un élément HTML

1. Accès direct

L'objet **Document** possède de nombreuses propriétés et méthodes. Cinq de ces dernières permettent de « pointer » directement un ou plusieurs éléments dans le document.

- La méthode **getElementById()** ;
- La méthode **getElementsByTagName()** ;
- La méthode **getElementsByClassName()** ;
- La méthode **querySelector()** ;
- La méthode **querySelectorAll()**.

Accéder a un élément HTML

1. Accès direct

➤ La méthode `getElementById()` :

La méthode, `getElementById()`, va nous permettre de cibler un élément HTML possédant un attribut **id** en particulier. C'est certainement la méthode la plus utilisée.

- Si l'élément est trouvé, `getElementById()` va renvoyer **l'élément** en tant qu'objet.
- Si aucun élément n'est trouvé, la méthode renverra la valeur **null**.

```
let x = document.getElementById("demo");
```

Accéder a un élément HTML

1. Accès direct

➤ La méthode `getElementsByName()` :

La méthode `getElementsByName()` va retourner des informations relatives à tous les éléments HTML d'un même « genre » **dans un tableau**.

- Cette méthode va prendre le nom du type d'élément à récupérer en **argument**.
- L'intérêt de cette méthode est qu'on va ensuite pouvoir récupérer un élément en question en utilisant un **indice** du tableau créé.

```
let y = document.getElementsByName("p");
```


Accéder a un élément HTML

1. Accès direct

➤ La méthode `getElementsByClassName()` :

La méthode `getElementsByClassName()` va nous permettre d'accéder aux éléments HTML disposant d'un attribut **class** en particulier.

- `getElementsByClassName()` va prendre la valeur d'un attribut **class** en **argument**.
- Elle va s'utiliser de manière similaire à la méthode précédente et renvoyer également **un tableau**..

```
let y = document.getElementsByClassName("class1");
```

Accéder a un élément HTML

1. Accès direct

Pour faciliter la sélection d'éléments suivant des critères complexes, le DOM s'est enrichi de deux nouvelles méthodes. La première, nommée **querySelectorAll**, permet de rechercher des éléments à partir d'un sélecteur CSS. autre méthode de recherche d'éléments à partir d'un sélecteur CSS s'appelle **querySelector**. Elle fonctionne de manière identique à querySelectorAll, mais renvoie uniquement le premier élément correspondant au sélecteur passé en paramètre.

Accéder a un élément HTML

1. Accès direct

➤ La méthode `querySelector()` :

La méthode `querySelector()`, qui prend en argument une **expression CSS**, permet de cibler directement **le premier** élément d'un ensemble de nœuds.

```
let x = document.querySelector(".class1");  
let y = document.querySelector("#p1");  
let z = document.querySelector("p,h1");  
let w = document.querySelector("div p");
```

Accéder a un élément HTML

1. Accès direct

➤ La méthode `querySelectorAll()` :

La méthode `querySelectorAll`, qui prend en argument une **expression CSS**, permet de cibler **tous les éléments** d'un ensemble de nœuds.

- **`querySelector()`** va renvoyer des informations relatives au premier élément trouvé correspondant au sélecteur CSS sélectionné, tandis que **`querySelectorAll()`** va renvoyer des informations sur tous les éléments correspondants.
- **un tableau** va être créé lorsqu'on utilise `querySelectorAll()`.

Accéder a un élément HTML

1. Accès direct

➤ La méthode `querySelectorAll()` :

```
let x = document.querySelectorAll (".class1");  
let y = document.querySelectorAll ("#p1");  
let z = document.querySelectorAll ("p,h1");  
let w = document.querySelectorAll ("div p");
```

- 1. [Définition DOM](#)
- 2. [La structure d'une page web](#)
- 3. [L'objet Document](#)
- 4. [Accéder a un élément HTML](#)
- 5. [Modifier du contenu HTML](#)

Accéder a un élément HTML

1. Accès direct

Exemple:

```
let x = document.getElementById("demo");  
let y = document.getElementsByTagName("p");  
let z = document.getElementsByClassName("intro");  
let a = document.querySelector("p.intro");  
let b = document.querySelectorAll("p.intro");
```

Accéder a un élément HTML

2. Accès relatif

Les méthodes précédentes demandent de connaître précisément soit l'identifiant, soit le nom exact du nœud cherché. Nous allons voir maintenant comment il est possible d'accéder à des collections de nœuds dont on ne connaît pas ces caractéristiques a priori.

- **Naviguer entre les noeuds**

Accéder a un élément HTML

2. Accès relatif

- **Naviguer entre les noeuds**

Vous pouvez utiliser les propriétés de nœud suivantes pour naviguer entre les nœuds avec JavaScript:

- **parentNode**
- **childNodes[*nodenumber*]**
- **firstChild**
- **lastChild**
- **lastElementChild**
- **nextSibling**
- **previousSibling**

1. [Définition DOM](#)
2. [La structure d'une page web](#)
3. [L'objet Document](#)
4. [Accéder a un élément HTML](#)
5. [Modifier du contenu HTML](#)

Accéder a un élément HTML

2. Accès relatif

```
let y = X.parentNode
```

renvoie le nœud-parent de l'élément `X`.

```
let x = element1.childNodes;
```

donne la liste de tous les nœuds-enfants de l'élément `élément1` sous la forme d'un tableau.

```
element1.childNodes[0]
```

renvoie le premier nœud-enfant de l'élément `élément1`

- 1. [Définition DOM](#)
- 2. [La structure d'une page web](#)
- 3. [L'objet Document](#)
- 4. [Accéder a un élément HTML](#)
- 5. [Modifier du contenu HTML](#)

Accéder a un élément HTML

2. Accès relatif

element1. firstChild

est équivalent à **élément1.childNodes[0]**, et renvoie le premier nœud-enfant de l'élément **élément1** .

element1. lastChild;

renvoie le dernier nœud-enfant de l'élément **élément1**

element1. lastElementChild;

renvoie le dernier élément-enfant de l'élément **élément1**

1. [Définition DOM](#)
2. [La structure d'une page web](#)
3. [L'objet Document](#)
4. [Accéder a un élément HTML](#)
5. [Modifier du contenu HTML](#)

Accéder a un élément HTML

2. Accès relatif

element1. previousSibling

Le nœud courant peut avoir des frères. On accède au frère précédent à l'aide de la propriété `previousSibling`

element1. nextSibling

On accède au frère suivant par la propriété `nextSibling`

1. [Définition DOM](#)
2. [La structure d'une page web](#)
3. [L'objet Document](#)
4. [Accéder a un élément HTML](#)
5. [Modifier du contenu HTML](#)

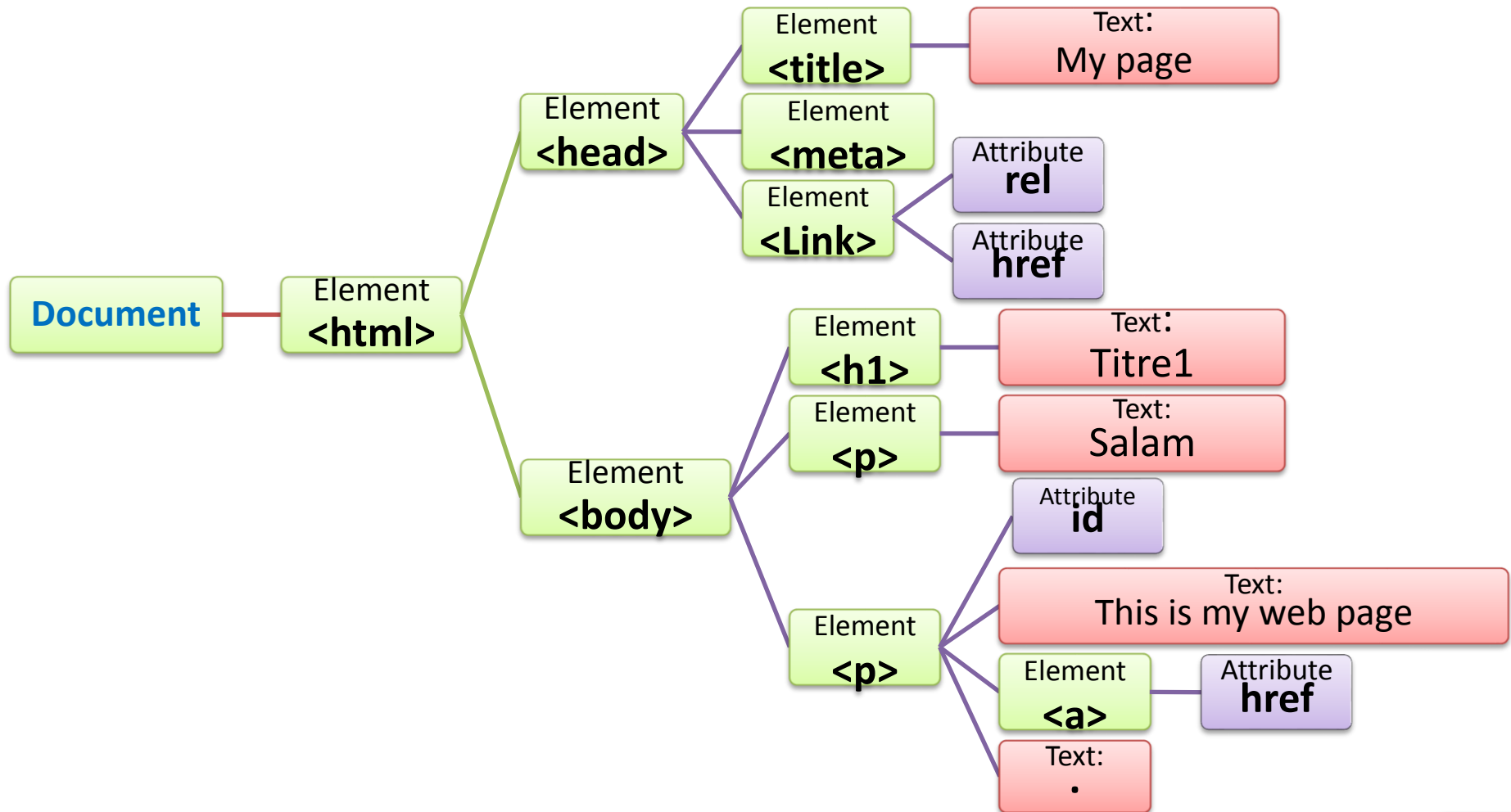
Accéder a un élément HTML

Example

```
<html>
<head>
  <title>My page</title>
  <meta charset="utf-8" />
  <link rel="stylesheet" href="design.css" />
</head>
<body>
  <h1>Titre1</h1>
  <p>Salam,</p>
  <p id='demo'>This is my web page<a href="http://mypage.com">Here</a>.</p>
</body>
</html>
```

1. [Définition DOM](#)
2. [La structure d'une page web](#)
3. [L'objet Document](#)
4. [Accéder a un élément HTML](#)
5. [Modifier du contenu HTML](#)

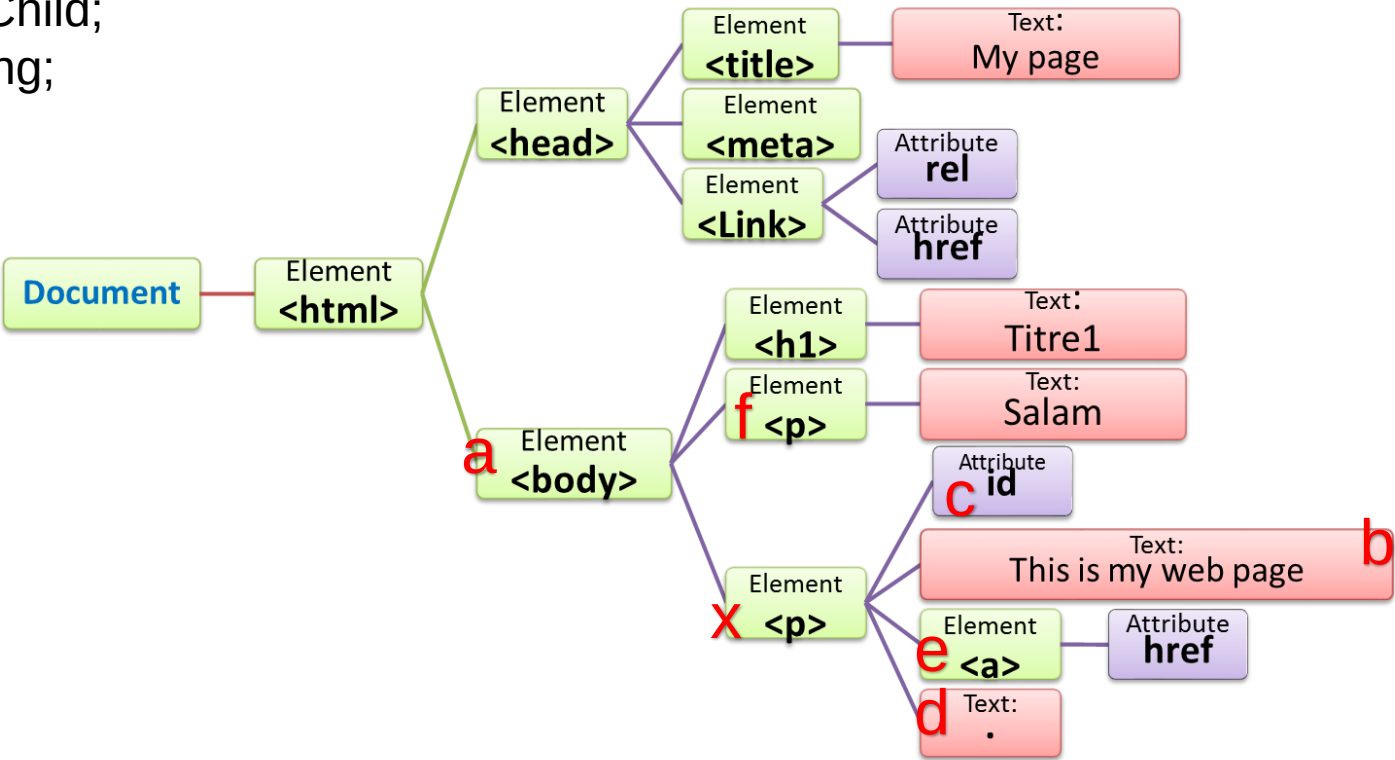
Accéder a un élément HTML



- 1. Définition DOM
- 2. La structure d'une page web
- 3. L'objet Document
- 4. Accéder a un élément HTML
- 5. Modifier du contenu HTML

Accéder a un élément HTML

```
var x = document.getElementById("demo");  
var a:=x.parentNode;  
var b:=x. childNodes[1];  
var c:=x. firstChild;  
var d:=x. lastChild;  
var e:=x. lastElementChild;  
var f:=x. previousSibling;  
var g:=x. nextSibling;
```



g → null

1. [Définition DOM](#)
2. [La structure d'une page web](#)
3. [L'objet Document](#)
4. [Accéder a un élément HTML](#)
5. [Modifier du contenu HTML](#)

Accéder a un élément HTML

2. Accéder directement à des éléments particuliers

Finalement, l'objet **Document** fournit également des propriétés qui vont nous permettre d'accéder directement à certains éléments ou qui vont retourner des objets contenant des listes d'éléments.

body, head, links, title, url, scripts, cookie

```
document.body.style.color = 'blue';
```

Accéder a un élément HTML

2. Accéder directement à des éléments particuliers

Les propriétés qui vont le plus nous intéresser ici sont les suivantes :

- La propriété **body** qui retourne le nœud représentant l'élément body ;
- La propriété **head** qui retourne le nœud représentant l'élément head ;
- La propriété **links** qui retourne une liste de tous les éléments a ou area possédant un href avec une valeur ;
- La propriété **title** qui retourne le titre (le contenu de l'élément title) du document ou permet de le redéfinir ;
- La propriété **url** qui renvoie l'URL du document sous forme de chaine de caractères ;
- La propriété **scripts** qui retourne une liste de tous les éléments script du document ;
- La propriété **cookie** qui retourne la liste de tous les cookies associés au document sous forme de chaine de caractères ou qui permet de définir un nouveau cookie.

1. [Définition DOM](#)
2. [La structure d'une page web](#)
3. [L'objet Document](#)
4. [Accéder à un élément HTML](#)
5. [Modifier du contenu HTML](#)

Modifier du contenu HTML

1. Modifier le contenu d'un élément HTML

La propriété **innerHTML** va également nous servir à modifier le contenu d'un élément HTML. Pour cela, il suffit d'utiliser **innerHTML** sur un élément et de lui affecter une nouvelle valeur textuelle

`<p id='demo'>Salam</p>`

Salam

```
document.getElementById("demo").innerHTML += ' world';
```

Salam world

Modifier du contenu HTML

2. Modifier la valeur d'un attribut HTML

Pour modifier la valeur d'un attribut HTML, nous allons généralement utiliser la propriété **attribute** de l'objet Element. Pour utiliser cette propriété, nous n'allons pas écrire **attribute** mais bien **le nom de l'attribut** donc on souhaite modifier la valeur (comme **href** par exemple). Ensuite, nous n'aurons plus qu'à affecter une nouvelle valeur à l'attribut ciblé.

```
<p>This is my web page<a href="http://mypage.com">Here</a>.</p>
```

```
document.querySelector("p a").href='http://www.mypage.dz';
```

- 1. [Définition DOM](#)
- 2. [La structure d'une page web](#)
- 3. [L'objet Document](#)
- 4. [Accéder à un élément HTML](#)
- 5. [Modifier du contenu HTML](#)

Modifier du contenu HTML

2. Modifier la valeur d'un attribut HTML

Notez également que dans le cas d'attributs non standards (ce qui est très rare), nous utiliserons alors plutôt les méthodes `getAttribute()` (pour accéder à l'attribut) et `setAttribute()` (pour modifier un attribut).

```
<p>This is my web page<a href="http://mypage.com">Here</a>.</p>
```

```
document.querySelector("p a").href='http://www.mypage.dz';
```



```
var x=document.querySelector("p a");  
x.setAttribute("href", 'http://www.mypage.dz')
```

1. [Définition DOM](#)
2. [La structure d'une page web](#)
3. [L'objet Document](#)
4. [Accéder à un élément HTML](#)
5. [Modifier du contenu HTML](#)

Modifier du contenu HTML

3. Modifier le CSS d'un élément HTML

Pour ajouter ou modifier le style CSS d'un élément HTML, on va utiliser la propriété **style** de l'objet Element suivie de **la propriété** CSS à ajouter ou modifier.

```
<body>
  <h1>Titre1</h1>
  <p>Salam,</p>
  <p>This is my web page<a href="http://mypage.com">Here</a>.</p>
</body>
```

```
document.querySelector("p").style.color='green';
document.querySelector("p").style.fontSize='35px';
```

1. [Définition DOM](#)
2. [La structure d'une page web](#)
3. [L'objet Document](#)
4. [Accéder à un élément HTML](#)
5. [Modifier du contenu HTML](#)

Modifier du contenu HTML

3. Modifier le CSS d'un élément HTML

```
<body>
  <h1>Titre1</h1>
  <p>Salam,</p>
  <p>This is my web page<a href="http://mypage.com">Here</a>.</p>
</body>
```

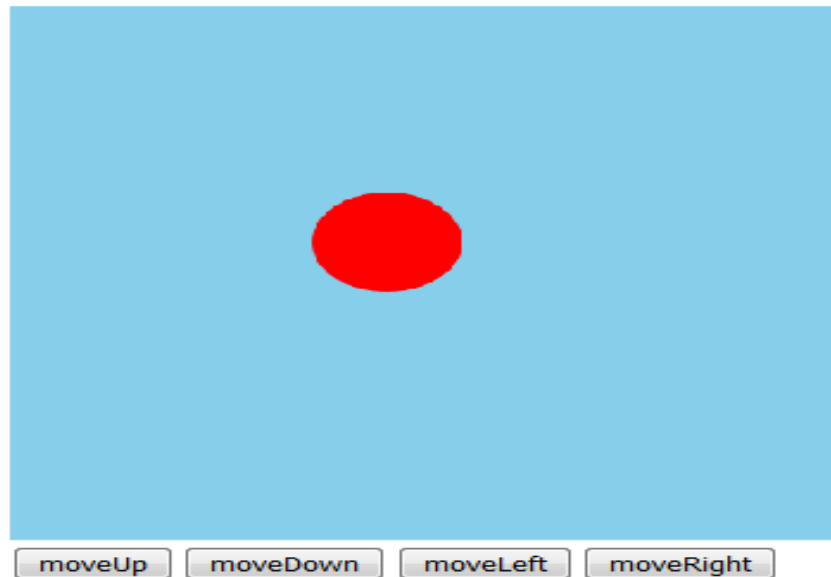
```
var x=document .querySelectorAll(" p,h1,h3 " );

for (var i=0;i<x .length ;i++) {
  x[i].style. font-family =" arial, sans-serif";
  x[i].style. color ="#FFCC66"}
```

TP3 Exercice 1

utilisez la technologie SVG et JavaScript pour réaliser le programme suivant, Une page Web avec :

- une zone graphique (400x400) contenant un cercle rouge ($r=40$) au milieu.
- et quatre boutons pour déplacer le cercle vers le haut, le bas, la droite et la gauche.



```
<svg width="440" height="440" style="background-color:skyblue">  
  <circle r="40" cx="200" cy="200" fill="red" id="circl1" />
```

Sorry, your browser does not support inline SVG.

```
</svg>
```

```
<br>
```

```
<button onclick="moveUp();">moveUp</button>
```

```
<button onclick="moveDown();">moveDown</button>
```

```
<button onclick="moveLeft();">moveLeft</button>
```

```
<button onclick="moveRight();">moveRight</button>
```

```
<br>
```

```
let circl=document.getElementById('circl1');
let cx =Number(circl.getAttribute('cx'));
let cy =Number(circl.getAttribute('cy'));

function moveUp(){
    cy =cy-10; circl.setAttribute('cy',cy-10);
}
function moveDown(){
    cy =cy+10; circl.setAttribute('cy',cy-10);
}
function moveLeft(){
    cx =cx-10; circl.setAttribute('cx',cx-10);
}
function moveRight(){
    cx =cx+10; circl.setAttribute('cx',cx-10);
}
```



```
<svg width="440" height="440" style="background-color:skyblue">  
  <circle r="40" cx="200" cy="200" fill="red" id="cirl1" />  
  Sorry, your browser does not support inline SVG.  
</svg>
```

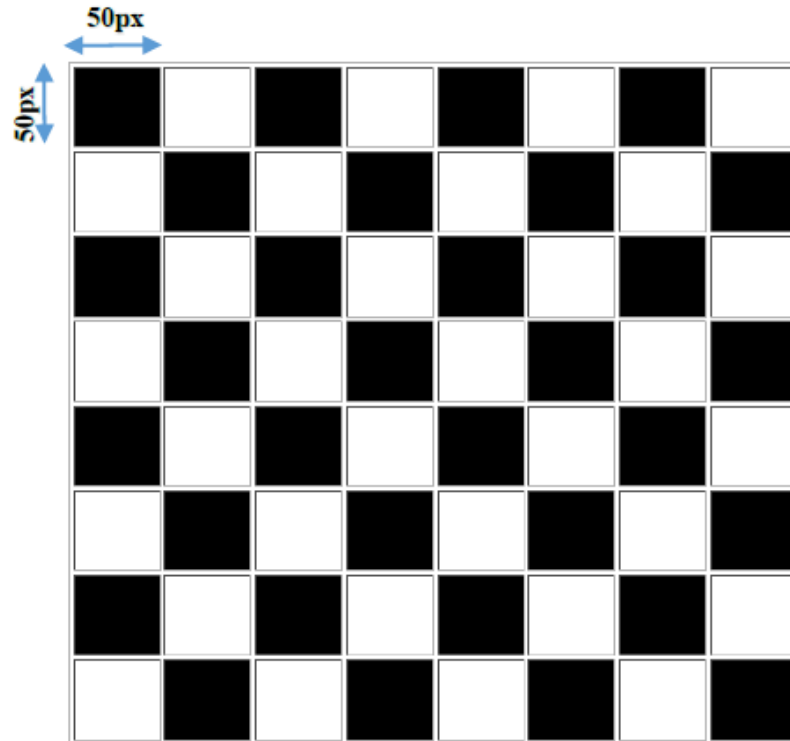
```
<br>  
  <button onclick="move(0,-10);">moveUp</button>  
  <button onclick="move(0,10);">moveDown</button>  
  <button onclick="move(-10,0);">moveLeft</button>  
  <button onclick="move(10,0) ;">moveRight</button>  
<br>
```

```
let cer=document.getElementById('circ11');

function move(x1,y1){
  let x=Number(cer.getAttribute('cx'));
  let y=Number(cer.getAttribute('cy'));
  cer.setAttribute('cy',y+y1);
  cer.setAttribute('cx',x+x1);
}
```

TP3 Exercice 4

Écrivez un script JavaScript qui dessine un échiquier ; en utilisant l'élément HTML `<table>`.



TP3 Exercice 4

Note:

Les nouveaux noeuds DOM sont créés à l'aide de la méthode **document.createElement(X)**. en utilisant le nom de balise fourni en paramètre ;

La méthode **appendChild(Y)** ajoute un nouvel enfant.

Exemple : Le code suivant ajoute un nouvel élément de liste () à la fin de la liste:

```
<ul id="list"><li>Item</li></ul>

<script>
  let list = document.getElementById("list");
  let newNode = document.createElement("li");
  list.appendChild(newNode);
</script>
```

```
<script >
  let tbl = document.createElement('table');
  tbl.setAttribute('border', '1');
  let tbdy = document.createElement('tbody');
  for (let i = 0; i < 8; i++)
  {
    let tr = document.createElement('tr');
    tr.setAttribute('height', '50px');

    for (var j = 0; j < 8; j++)
    {
      let td = document.createElement('td');
      if((i+j)%2 == 0)
      {
        td.style.background = 'black';
      }
      td.setAttribute('width', '50px');
      tr.appendChild(td)
    }
    tbl.appendChild(tr);
  }
  document.body.appendChild(tbl)
</script>
```